

The Procurement Exception Command Center

From one disruption to a governed, self-driving recovery operation.

Outcome: cost avoided and time-to-recovery across concurrent disruptions

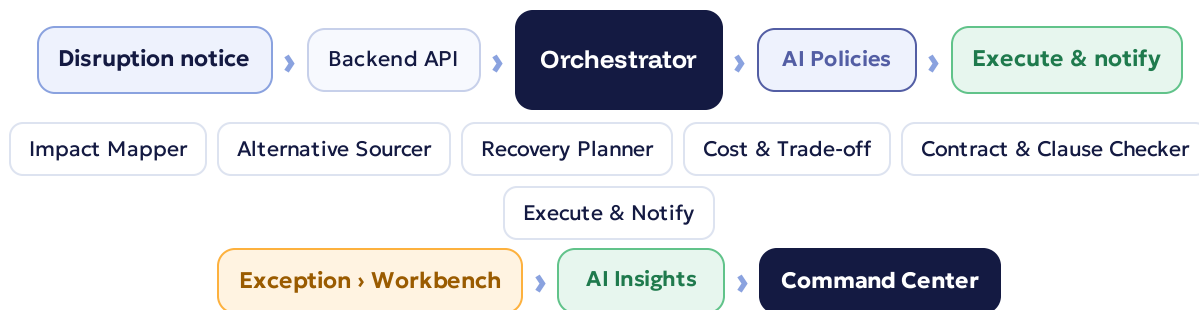
Round 1 track: The Procurement Exception Commander

What changed since Round 1. Your Round 1 AI Employee is still the brain and is not thrown away. Round 2 puts a real operation around it: the agent grows to five or more Operators on Auto, and you build a coded Command Center on the template with AI Policies, AI Insights, an AI Manager, a Data Manager, and a Workbench. The bar moves from "can you decompose the problem" to "can you run a governed operation."

THE SCENARIO

Wei no longer firefights one alert; he runs supply-chain exception management across tier-1, tier-2, and tier-3 suppliers, several warehouses, in-transit shipments with port cut-offs, and downstream customer orders that depend on all of it. A tier-2 supplier fails and quietly takes three tier-1 suppliers with it. A demand spike and a port strike land in the same hour, both wanting the same alternative supplier and the same buffer stock. An expedite would fix one line but breaches a contract escalation clause, and may still be cheapest once penalties are counted. The Round 2 build holds several disruptions at once, weighs them against each other, and routes the high-stakes calls to a human commander.

HOW THE AI EMPLOYEE WORKS (EXAMPLE)



Impact and options are assessed in parallel, the Orchestrator branches on severity, and it holds state when several disruptions compete for the same resource.

This is one example, not a required design. Add or remove Operator Agents, change how each one works and the logic behind it, make the flow more optimal, and make sure it handles every kind of exception. Teams differ on the how; judges compare on the result.

WHAT RUNS WHERE

LAYER	RUNS ON	WHAT IT IS
The agent	Auto	One Orchestrator plus five or more Operator Agents doing the actual supply-chain work. Orchestration must be built on Supervity Auto.
Command Center	Template	The live operational picture: KPIs, agent status, exception queues.
AI Policies	Template	Configurable business rules that constrain what the agent may do alone.
AI Insights	Template	Patterns, anomalies, and recommendations from the operational data.
AI Manager	Template	Chat and orchestration surface that talks to the Auto agent.
Data Manager	Template	Live registry of every connected system and its health.
Workbench	Template	The human queue where exceptions are reviewed and cleared.

Start from the template: github.com/digitamizers/AutoPilot-Template. Clone it and run `docker compose up` for the backend, database, and Command Center shell. Generate your Workflow API key at auto.supervity.ai/u/api-keys and read auto.supervity.ai/docs before wiring. Full walkthrough in the Round 2 Participant Guide.

WHAT YOU BUILD

The agent on Auto

One Orchestrator plus five or more Operators doing the real work. Extend your Round 1 build.

Command Center

Live view of open disruptions by severity, cost at risk versus cost avoided, time-to-recovery, service level or fill rate, supplier health, and inventory position across nodes.

AI Policies engine

At least three editable rules: severity thresholds for auto-action versus escalation, expedite spend limits, contract escalation clauses that block an expedite, substitution eligibility, and customer-tier priority.

AI Insights

Recurring delay patterns by supplier, purchase orders trending at risk, demand anomalies beyond safety stock, single-source exposure, and contracts expiring into a risky window.

AI Manager

A chat and orchestration surface where a person asks the operation questions and watches the Orchestrator delegate to Operators and report back.

Data Manager

A live registry of every connected system, what each is used for, and whether the connection is healthy.

The Workbench

The commander queue for expensive expedites, clause breaches, customer-facing reallocations, and resource conflicts between disruptions, each with the trade-off analysis.

WHAT YOU CAN ADD IN ROUND 2

Round 1 covered the core path. These are the business functions this operation is still missing, and each one is naturally its own Operator. Pick the ones that make your build a fuller supply-chain AI Employee; you are not required to build all of them.

Multi-event prioritisation

Rank concurrent disruptions that compete for the same buffer stock or the same alternative supplier.

RFQ and quote comparison

Request and compare alternative supplier quotes on price, lead time, and minimum order quantity.

Inventory reallocation across nodes

Move stock between warehouses to protect priority orders and the highest-value customers.

Customer re-promising and comms

Recalculate promise dates and notify affected customers before they chase you.

Penalty and clause modelling

Score each expedite option against contract penalties and escalation clauses before acting.

Supplier scorecard

Maintain rolling supplier reliability and flag single-source and sole-source exposure.

WORKING WITH THE DATA

The dataset is how you exercise your integrations, not a file to read from. Either seed the systems you connect with it (load the records into your CRM, ticket tool, database, or an Airtable base), or park it in a store like SharePoint, OneDrive, Box, or Dropbox and point an integration at it, then pull the data back through that integration. Either way the data reaches your Operators through a live integration, at least three: one channel, one system of record, and a live exception to a person.

A way in

A disruption notice via email or Teams.

A system of record

Orders and inventory in Airtable or Supabase; tasks in Jira or Asana.

A human loop

A Slack or Teams approval to the commander before any high-impact action, routed through the Workbench.

Note: Google integrations (Drive, Sheets, Gmail, Calendar) are in beta for this event, so use alternatives such as Airtable, Supabase, SharePoint, OneDrive, Box, Dropbox, and Outlook.

THE ROUND 2 DATA PACK

WHAT IS IN IT

Fourteen tables and roughly 1,850 rows of Coupa-style procurement data: suppliers, contracts, purchase order headers and lines, order confirmations, inventory positions, demand signals, and disruption notices, plus the Round 2 additions: Supplier_Tiers, Warehouses, Shipments, Customer_Orders, Alternative_Suppliers, and Penalties.

WHAT IS NEW SINCE ROUND 1

Six new tables: Supplier_Tiers, Warehouses, Shipments, Customer_Orders, Alternative_Suppliers, and Penalties, covering tier dependencies, multi-node stock, in-transit freight, downstream orders, sourcing options, and contract penalties. The pack keeps the Round 1 schema, so your existing build still reads it, and adds volume and harder cases. Build for the general case: a judge may ask you to run a record you did not prepare.

MANDATORY IN ROUND 2

- 1. Agent on Auto.** One Orchestrator coordinating at least five distinct Operators, with parallel, branching, or stateful behavior.
- 2. Connected Command Center.** The coded frontend is wired to the Auto agent through the backend API and shows live agent activity, not the template's demo data.
- 3. Working AI Policies.** At least three active policies that measurably change agent behavior, editable without code, with every evaluation logged.
- 4. A live human loop.** At least one real exception routed to the Workbench with full context and resolved there.
- 5. Live integrations.** At least three across two categories (one channel, one system of record), visible and healthy in the Data Manager.

DO'S AND DON'TS

Do

- Build the Operators on Auto; code the Command Center, Policies, Insights, Manager, and Data Manager around them.
- Make policies genuinely dynamic: a changed threshold changes what the agent does, live.
- Generate insights from the data the agent actually processed.
- Route real exceptions to the Workbench with full context and a recommendation.

Don't

- Rebuild the Orchestrator or Operators outside Supervity Auto.
- Ship policy or insight pages that only show the template's static demo data.
- Let the agent invent a value on a missing field; pause and escalate.
- Hardcode to these rows, skip the human loop, or stage the demo.

Bonus. Going beyond the brief earns bonus points once the stated problem is solved: extra Operators and richer downstream actions, forecasting, self-learning where a human correction at the Workbench changes future behavior, deeper auditability, meaningful use of open-source components given a real role in the build, and genuine creativity on top of the problem statement.

Not apps. Not agents. AI Employees run the Autos.